PRODUCT & ENGINEERING BRIEF

SCHOOL MANAGEMENT PLATFORM

How the system works and why it is built this way

A practical overview of the product, architecture, key flows, engineering decisions, and the trade-offs made during implementation.

In one sentence

A multi-school web application for managing people, teaching, academic records, communications, billing, and school subscriptions, with role- and school-scoped access.

Prepared: 2 November 2026
Scope: Repository structure plus implementation history available in this work session
Handling: No environment values, passwords, API keys, or database credentials are included.

01 / EXECUTIVE OVERVIEW

What this project is

The product is a school operations platform with a NestJS/TypeScript API, a React/Vite browser client, and a PostgreSQL database accessed through Prisma. Its central design problem is not only managing school records: it is ensuring each user can access only the right school, role, student, class, and financial data.

People and access

School admins manage staff, teachers, parents, students, invitations, memberships, and roles. Teachers use admin-created accounts; students and parents have scoped self-service access.

Academic operations

Academic sessions and terms organize classes, subjects, enrollments, attendance, timetables, examinations, and results.

School administration

Announcements, notifications, audit history, dashboards, school profile and subscription-plan limits support day-to-day operations.

Money flows

Fee structures, invoices, payments and subscription billing connect to Paystack, with server-side verification and lifecycle handling.

How to read this brief

This is a product and engineering overview, not a claim that every route or business rule has been freshly audited line by line. It combines the current repository's main structure with recorded implementation and validation history. Anything requiring an external account, deployment environment, or live provider configuration is called out rather than assumed complete.

> **Project emphasis:** tenant isolation, safer authentication/session lifecycle, role-specific privacy, and transaction-safe academic/financial operations.

Source basis: backend module tree, Prisma schema, package manifests, repository documentation, and implementation history. Secrets and the local environment file were excluded.

02 / ARCHITECTURE

Application structure

The system is a modular monolith: one API process contains separate NestJS feature modules, shares a Prisma/PostgreSQL data layer, and serves requests from a separate single-page frontend. This keeps local development and feature integration straightforward while preserving clear domain boundaries.

Layer	Technology / responsibility	Why it fits this stage
Browser app	React 19, TypeScript, Vite, React Router	Single client for login, dashboards, academic workflows, forms, and role-based navigation.
API	NestJS 12, TypeScript, Express adapter	Controllers, DTO validation, decorators, guards, dependency injection, and feature modules give the backend structure.
Persistence	PostgreSQL, Prisma 7, PostgreSQL driver adapter	Relational constraints represent school membership and academic/financial relationships; Prisma provides typed access and migrations.
Authentication	JWT access tokens, database-backed refresh tokens, bcrypt	Short-lived access tokens with revocable, rotated sessions balance stateless request authentication with session control.
External services	Paystack payments; SMTP mail via Nodemailer; hosted Neon database in the current dev setup	Delegates payment processing, email transport and hosted database operations to established providers.

Backend feature areas

Auth & membership · Schools & onboarding · Students & parents · Teachers · Classes & subjects · Sessions & terms · Attendance · Exams & results · Fees & invoices · Payments & subscriptions · Announcements & notifications · Dashboard & audit log

> **Data model shape:** schools own academic/operational records; users join schools through memberships. Profiles such as teacher, student, and parent connect person identity to school-specific roles and records. Unique constraints and indexes support membership and common school-scoped lookups.

Local development topology

The API and frontend run as separate development servers. For phone testing over a LAN, the browser must call the development computer's LAN address rather than `localhost` (which points to the phone). The API's CORS allowlist must include the exact frontend origin, and the host firewall/network must allow traffic.

Current runtime constraints: Node.js is constrained by the backend package manifest; local frontend and API environment values are intentionally not reproduced here.

03 / AUTHENTICATION AND AUTHORIZATION

Who can sign in—and what they can see

Account lifecycle

1. **School registration:** registration creates a school, a school-admin user, and the membership linking them in a database transaction. School name and address belong to the School record; the login identity remains a normal user email.
2. **Email verification:** normal registration issues a short-lived, single-use verification token and sends a link. Resend behavior avoids revealing whether a given email is registered.
3. **Sign-in and lockout:** users authenticate with email and password. Failed attempts are counted; five failed attempts trigger a temporary 15-minute lockout. Authentication endpoints also have request throttles.
4. **Session lifecycle:** access tokens are short lived (15 minutes in the reviewed implementation); refresh tokens are stored as hashes, rotated on use, and grouped into session families. Replay detection can revoke a family. Logout and password changes/reset invalidate sessions.
5. **Teacher provisioning:** an authorized school administrator creates the teacher account. A temporary password is sent through email; the teacher must change it before using ordinary protected routes.

Authorization layers

- **JWT guard:** validates the access token and checks current account/session state.
- **Roles guard and decorator:** restrict endpoints to intended user roles.
- **School membership/access checks:** bind a request to a school membership, rather than trusting a client-supplied school ID.
- **Service-level ownership scope:** students see their own records; parents see their linked children; teachers see assigned classes; finance and administrator records are restricted to permitted roles.

> **Defense-in-depth choice:** role checks say what kind of action a user can take; tenant and record ownership checks say which school's or person's data they can act on. Both are needed. A role-only policy is not enough for a multi-school system.

Privacy rules encoded in workflows

Unpublished exam results are withheld from student/parent self-service. Exam status transitions are constrained and audited. Class rosters, payment/invoice data, audit records, dashboards, announcements and notification recipients are filtered by role, membership, audience, or ownership. Responses avoid embedding unnecessary related records such as complete teacher/user profiles or full invoice/payment collections.

> **Development-only exception:** a configuration switch can bypass email verification only when the runtime is explicitly in development. It lets testers register without a working email sender; it must stay off in production. It bypasses email verification—not every email-dependent feature.

Security posture is layered, not absolute: deployment configuration, database availability, provider settings, secrets, and regression tests remain part of the security boundary.

04 / PRODUCT WORKFLOWS

Academic and school operations

People and onboarding

School membership and invitations organize users by school and role. Teacher creation and class/subject assignment are explicit admin workflows. Student and parent profiles are linked to login identities where appropriate.

Classes and enrollment

Classes are associated with academic sessions. Enrollment paths enforce class ownership and capacity rules. Student status transitions such as graduation or transfer are reflected in active enrollment behavior.

Teaching operations

Teachers can be assigned to subjects/classes; a class can designate a primary teacher. Timetable assignments protect schedules from being orphaned by unassignment.

Attendance and results

Attendance is class-scoped. Exams follow a controlled lifecycle; result publication governs when student and parent self-service can display scores.

Communication

Announcements and notifications use audience and recipient restrictions. Sensitive management and recipient-directory operations are not treated as public school-wide reads.

Visibility and reporting

Dashboard aggregates follow the caller's role, school, children, or class assignments. Revenue and broad school-wide aggregates are limited to appropriate staff roles.

Academic calendar decisions

Terms are checked for overlap and tied to sessions; active term handling is updated when the academic session changes. A session rollover capability was implemented in the recorded work, to carry forward appropriate academic structure rather than silently mixing records between sessions. Rollover is a deliberate operation because copying classes/enrollments is consequential and must be reviewable.

Data integrity philosophy

Where a hard delete could cascade into results, attendance, invoices, or payments, guarded deletion or lifecycle status changes are preferred. This reduces irreversible loss and preserves a useful historical trail. Existing relational cascades still need to be considered when designing future delete operations.

> **Trade-off:** explicit assignment, publication, and rollover steps add work for administrators, but make school-year boundaries and the release of student results intentional instead of implicit.

05 / BILLING AND PAYMENTS

Money flows and safeguards

School fees

The application models fee structures, invoices, student payments, and payment status. Paystack is used to initialize payment and verify provider results; webhook signatures are checked. Server-side checks constrain who can see or initiate payment for a student's invoice. Pending-payment reservations and amount caps reduce the chance that concurrent checkouts or manual status changes permit overpayment. Late provider callbacks are guarded so they do not casually rewrite a final/refunded state.

Subscriptions

Schools have subscription plans and payment state. Plan limits are checked against managed counts before relevant operations. Scheduled lifecycle work expires old pending checkouts and subscriptions whose paid period has ended, rather than requiring an operator to update every record manually.

Settlement routing

Per-school settlement was added through Paystack subaccount support. The school configures settlement details with Paystack; the app stores the provider's subaccount identifier rather than retaining the bank account number. Fee checkout can route through that configured subaccount. Provider activation and valid account ownership still depend on Paystack onboarding and environment credentials.

Financial correctness choices

- Use database transactions/conditional updates for state transitions that must not race.
- Use high-entropy references and collision-aware invoice numbering to reduce predictable identifiers and race failures.
- Prefer provider verification/webhook evidence over trusting browser redirects.
- Restrict invoice/payment detail access to owners and finance roles.
- Retain state history through statuses and audit events rather than silently overwriting final outcomes.

> **Provider dependency:** local/test configuration can validate request construction, but live settlement depends on the school's Paystack subaccount state, supported banking details, correct test/live keys, callback URLs and webhook configuration.

Payment provider interaction is external and can be delayed or unavailable. The application must treat provider responses as asynchronous, verify them, and present an honest pending/error state to users.

06 / CLIENT EXPERIENCE AND OPERATIONS

Frontend, email, and deployment behavior

Frontend behavior

The React client provides registration/login, password recovery, email verification, invitation acceptance, dashboards and school workflows. Shared API handling parses backend validation/error responses and presents more specific explanations for authentication, permission, missing-resource, throttling, timeout and server errors. Token refresh is centralized so ordinary pages do not each implement session renewal. Role-specific navigation complements—but does not replace—backend authorization.

Email and account links

SMTP transport is configured through environment variables. Email verification, password reset and invitation/teacher credential flows require a working sender. Resend's shared testing sender is constrained; without a verified domain, arbitrary recipients cannot be assumed deliverable. A real team test needs a permitted sender/recipient arrangement or a local mail catcher. The development verification bypass is a separate short-term testing option.

Cross-origin local testing

The API uses an explicit CORS origin allowlist with credentials enabled. Browser requests from phones must use the computer's LAN IP as the API host, and the CORS allowlist must match the phone-served frontend origin exactly. A CORS response is distinct from an API 503 or a Prisma connection failure; successful CORS headers do not prove the database or email provider is healthy.

Database connectivity and scheduled tasks

The app uses a PostgreSQL pool with connection timeout/keepalive settings. Recent diagnostics saw Prisma connection-closed/unreachable errors against the hosted Neon database, then verified DNS/port reachability and a simple Prisma query succeeding. This establishes that connectivity later recovered; it does not identify why the earlier interruption occurred. Subscription cleanup logs failures and retries on a later interval instead of crashing the process.

> **Useful diagnostic split:** CORS is a browser-origin policy; SMTP errors are mail transport/sender problems; Prisma P1001/P1017 errors are database connectivity. Each requires checking a different layer.

07 / TRADE-OFFS

Why these decisions—and what they cost

Decision	Benefit	Cost / limitation
Modular monolith instead of microservices	Simple deployment, shared transactions, faster product iteration, fewer network boundaries.	One process can become a scaling/release bottleneck; modules need discipline to avoid tight coupling.
Relational PostgreSQL + Prisma	Strong relations/constraints, typed queries, migrations and transactions suit academic and financial data.	Schema changes require migration planning; hosted database/network availability affects broad portions of the app.
Short access token + database-backed refresh session	Access credentials expire quickly; logout, password changes and replay detection can revoke sessions.	More database work on auth/session checks and more moving parts than a purely stateless JWT design.
Global guards plus service-level scoping	Layered controls reduce accidental access through a new route or nested relation.	More policy code and tests; every endpoint must keep its role, school and record ownership rules consistent.
Five-failure temporary lockout + throttling	Raises cost of password guessing and abuse.	Can inconvenience legitimate users and may be abused to lock out a known account; shared throttler storage is needed for consistent multi-instance limits.
Verified email by default; opt-in development bypass	Normal environment retains account ownership checks; test environment can onboard without a mail domain.	Bypass must be environment-gated and audited; it is unsafe if production is mislabeled as development or the flag is copied.
Temporary teacher password + forced change	Matches school-admin onboarding and makes initial credentials short-lived in use.	Emailing a temporary secret creates delivery and mailbox risk; a one-time setup link would be stronger long-term.
Paystack subaccounts for school settlement	Allows funds to settle to each school instead of one central account.	Requires provider onboarding, bank verification, compliance and correct test/live setup for every school.
Explicit session rollover and result publication	Prevents accidental mixing of academic periods or prematurely exposing scores.	Admin tasks take more steps and need clear previews/undo or correction procedures.
Soft status/guarded delete over broad hard delete	Preserves financial/academic history and reduces destructive cascades.	More lifecycle states, filters, retention rules and admin UX complexity.
In-process scheduled cleanup	Low setup cost and adequate for one API instance.	Not a durable distributed scheduler; multiple replicas may duplicate work, and process downtime delays tasks.

Decision	Benefit	Cost / limitation
In-memory throttling for current deployment	No additional infrastructure and straightforward local setup.	Limits reset per process/restart and do not coordinate across instances; use shared storage before horizontal scaling.

These are intentional engineering compromises for a product in active development, not endorsements that the present configuration is sufficient for every production scale or compliance regime.

08 / QUALITY AND CURRENT STATE

What has been verified—and what remains

Validation recorded during implementation

- Backend Nest build passed after recent database error handling and authentication changes.
- Frontend TypeScript/Vite production build passed after the registration and verification UX change.
- Focused tests passed for refresh-token rotation/replay, email verification, auth lockout/development bypass, payment state transitions, and database connection error responses at different points in the work.
- Targeted lint checks passed for recently changed modules; earlier whole-project lint had some unused-import warnings.
- Prisma migrations were deployed additively without resetting the database in the recorded work; the database was reported up to date then.
- Neon DNS and port reachability plus a simple query succeeded after an earlier logged connectivity failure; transient root cause was not established.

These checks were performed at different points as features changed. This document does not claim that a single clean full-suite CI run was performed against the exact repository state at PDF generation time.

Practical follow-up before public production use

1. **Rotate exposed credentials.** Credentials were present in an environment file shared during troubleshooting. If they are real, revoke and replace the mail/provider keys and JWT signing secret; do not put replacements in chats or source control.
2. **Keep the email bypass off in production.** Verify deployment environment and feature flag together, and test that production refuses the bypass.
3. **Use production-grade email.** Configure a verified sender/domain and test messages to independent recipient addresses; track bounces and delivery.
4. **Confirm Paystack live setup.** Verify each school subaccount and test initialization, webhook signatures, refunds/late callbacks, reconciliation and settlement.
5. **Plan scale operations.** Move throttling and scheduled lifecycle jobs to shared/distributed infrastructure if running multiple API replicas.
6. **Observe the database.** Review Neon connection limits/status and API logs/metrics; add alerting for repeated connection errors and failed lifecycle jobs.
7. **Run CI from a clean checkout.** Build backend and frontend separately, run all unit/e2e tests, validate/generate Prisma, and verify migration status before each release.

> **Do not treat the development setup as a production hardening checklist.** Real mail delivery, live payment settlement, reliable backups, secrets rotation, shared rate limiting, monitoring and restore testing all need explicit deployment ownership.

Bottom line

The core product has a broad school-operations model and has been strengthened around tenant isolation, authentication/session control, payment state and student privacy. The main trade-off is complexity in exchange

for tighter control and traceability. The next maturity step is operational reliability: shared infrastructure for multi-instance behavior, provider/onboarding validation, automated end-to-end testing and production observability.

End of report · Prepared 2 November 2026 · No secrets included